第 4 次上机

班级	学号	姓名

1、 中断程序设计

（1）反汇编的截图

```
-u
0780:000E B87E07        MOV     AX,077E
0780:0011 8ED8          MOV     DS,AX
0780:0013 B435          MOV     AH,35
0780:0015 B01C          MOV     AL,1C
0780:0017 CD21          INT     21
0780:0019 06            PUSH    ES
0780:001A 58            POP     AX
0780:001B A30E00        MOV     [000E],AX
0780:001E 891E0C00      MOV     [000C],BX
0780:0022 0E            PUSH    CS
0780:0023 58            POP     AX
0780:0024 BA0000        MOV     DX,0000
0780:0027 8ED8          MOV     DS,AX
0780:0029 B425          MOV     AH,25
0780:002B B01C          MOV     AL,1C
0780:002D CD21          INT     21
```

```
0780:002F B87E07        MOV     AX,077E
0780:0032 8ED8          MOV     DS,AX
0780:0034 B401          MOV     AH,01
0780:0036 CD21          INT     21
0780:0038 3C51          CMP     AL,51
0780:003A 75F8          JNZ     0034
0780:003C 8B160C00      MOV     DX,[000C]
0780:0040 A10E00        MOV     AX,[000E]
0780:0043 8ED8          MOV     DS,AX
0780:0045 B425          MOV     AH,25
0780:0047 B01C          MOV     AL,1C
0780:0049 CD21          INT     21
0780:004B B87E07        MOV     AX,077E
0780:004E 8ED8          MOV     DS,AX
```

```
0780:0050 A10A00          MOV     AX,[000A]
0780:0053 B90400          MOV     CX,0004
0780:0056 8D3E1000        LEA     DI,[0010]
0780:005A 51              PUSH    CX
0780:005B B104            MOV     CL,04
0780:005D D3C0            ROL     AX,CL
0780:005F 59              POP     CX
0780:0060 8AD8            MOV     BL,AL
0780:0062 80E30F          AND     BL,0F
0780:0065 80C330          ADD     BL,30
0780:0068 80FB39          CMP     BL,39
0780:006B 7603            JBE     0070
0780:006D 80C307          ADD     BL,07
```

```
0780:0070 881D            MOV     [DI],BL
0780:0072 47              INC     DI
0780:0073 E2E5            LOOP    005A
0780:0075 B409            MOV     AH,09
0780:0077 8D161000        LEA     DX,[0010]
0780:007B CD21            INT     21
0780:007D B8004C          MOV     AX,4C00
0780:0080 CD21            INT     21
```

（2）在进行计算前，显示 ID、NUM 的内存值的截图（多显示、少显示均扣分）

```
-d 0 b
077E:0000  32 32 33 36 31 31 34 39-36 38 00 00                2236114968..
```

（3）运行到返回 dos 前暂停，对屏幕显示的输出结果（NUM 值的对应的 ASCII 字符串）截图【结果要与步骤（4）中的内存值一致】

```
-g 007d
5
6
Q01DCh
```

（4）在完成步骤（3）操作后，立即显示 ID、NUM 的内存值的截图（多显示、少显示均扣分）

```
-d 0 b
077E:0000  32 32 33 36 31 31 34 39-36 38 DC 01                2236114968..
```

（5）源代码

```
; empty asm file
```

```
title I love asm

data segment
    ID      db   '2236114968'
    num     dw   0
    old_off dw   ?                ; 保存原中断向量偏移
    old_seg dw   ?                ; 保存原中断向量段地址
    hex_str db   4 dup(?), 'h$'    ; 显示字符串
data ends

code segment
    assume cs:code, ds:data

    count   proc  far
       push  ds
       push  ax
       mov   ax, data
       mov   ds, ax
       inc   num
       pop   ax
       pop   ds
       iret
    count   endp

    main    proc
        ; assign the data segment base address to DS
        mov   ax, data
        mov   ds, ax

        ; TODO ...
```

```
        ; | add your code between arrows |
        ; v ---------------------------- v
        mov   ah, 35h
        mov   al, 1Ch
        int   21h
        push  es
        pop   ax
        mov   old_seg, ax
        mov   old_off, bx

        ; 设置新 1Ch 向量指向 count
        push  cs
        pop   ax
        mov   dx, offset count
        mov   ds, ax
        mov   ah, 25h
        mov   al, 1Ch
        int   21h

        ; 恢复 DS 指向数据段
        mov   ax, data
        mov   ds, ax

wait_input:
        mov   ah, 01h
        int   21h
        cmp   al, 'Q'
        jne   wait_input
```

```
        ; 恢复原 1CH 中断向量
        mov   dx, old_off
        mov   ax, old_seg
        mov   ds, ax
        mov   ah, 25h
        mov   al, 1Ch
        int   21h
        mov   ax, data
        mov   ds, ax

        ; 将 NUM 转换为十六进制字符串
        mov   ax, num
        mov   cx, 4
        lea   di, hex_str
convert_loop:
        push  cx
        mov   cl, 4
        rol   ax, cl
        pop   cx
        mov   bl, al
        and   bl, 0Fh                   ; 取低 4 位
        add   bl, 30h                   ; 转换为 ASCII
        cmp   bl, 39h
        jbe   is_digit                  ; 0-9 直接跳转
        add   bl, 7                     ; A-F 额外加 7
is_digit:
        mov   byte ptr [di], bl
        inc   di
        loop  convert_loop
```

```
        ; 显示十六进制结果
        mov   ah, 09h
        lea   dx, hex_str
        int   21h
         ; ^ ---------------------------- ^
        ; |          The END             |

         ; method 2: return to dos
         mov   ax, 4c00h
         int   21h
    main    endp
code ends
end main
```

3、 BIOS 和 DOS 中断

（1）反汇编的截图

```
-u
0780:0000 B87E07        MOV     AX,077E
0780:0003 8ED8          MOV     DS,AX
0780:0005 8D1E1400      LEA     BX,[0014]
0780:0009 B90A00        MOV     CX,000A
0780:000C 8D3E0A00      LEA     DI,[000A]
0780:0010 B401          MOV     AH,01
0780:0012 CD21          INT     21
0780:0014 3C0D          CMP     AL,0D
0780:0016 7411          JZ      0029
0780:0018 3C30          CMP     AL,30
0780:001A 72F4          JB      0010
0780:001C 3C39          CMP     AL,39
0780:001E 77F0          JA      0010
```

```
0780:0020 2C30          SUB     AL,30
0780:0022 D7            XLAT
0780:0023 8805          MOV     [DI],AL
0780:0025 47            INC     DI
0780:0026 49            DEC     CX
0780:0027 75E7          JNZ     0010
0780:0029 B8004C        MOV     AX,4C00
0780:002C CD21          INT     21
```

（2）在进行计算前，显示 ID、BUFFER 的内存值的截图（多显示、少显示均扣分）

```
-d 0 13
077E:0000  32 32 33 36 31 31 34 39-36 38 00 00 00 00 00 00   2236114968......
077E:0010  00 00 00 00                                       ....
```

（3）输入回车后，显示 ID、BUFFER 的内存值的截图（多显示、少显示均扣分）

```
-d 0 13
077E:0000  32 32 33 36 31 31 34 39-36 38 07 05 09 01 03 06   2236114968......
077E:0010  08 00 02 04                                       ....
```

（4）源代码

```
; empty asm file

title I love asm

data segment
    ID      db  '2236114968'
   buffer   db  10 dup(?)
   table    db  7,5,9,1,3,6,8,0,2,4
data ends

code segment
    assume cs:code, ds:data
    main    proc
        ; assign the data segment base address to DS
        mov   ax, data
        mov   ds, ax
```

```
        ; TODO ...
        ; | add your code between arrows |
        ; v --------------------------- v
        lea   bx, table
        mov   cx, 10
        lea   di, buffer

input_loop:
        mov   ah, 01h
        int   21h

        ; 检查是否为回车符（0Dh）
        cmp   al, 0Dh
        je    end_input           ; 若输入回车，提前结束

        ; 过滤非数字字符
        cmp   al, '0'
        jb    input_loop
        cmp   al, '9'
        ja    input_loop

        sub   al, '0'             ; 将 ASCII 字符转换为数字（0-9）

        xlat                    ; 加密结果存入 AL

        mov   [di], al
        inc   di                ; 移动缓冲区指针

        dec   cx
```

```
       jnz   input_loop

end_input:
        ; ^ ---------------------------- ^
       ; |          The END             |

        ; method 2: return to dos
        mov   ax, 4c00h
        int   21h
    main    endp
code ends
end main
```